

Dynamic Configuration System

.NET 8 ile yazılmış, MongoDB storage, in-memory cache ve otomatik arka plan yenileme desteği sunan dinamik konfigürasyon yönetim sistemi

.NET 8

MongoDB

RabbitMQ

Clean Architecture

39 Unit Test

İçindekiler

1. Proje Genel Bakışı

2. Hızlı Başlangıç

3. Mimari Yapı

4. Bileşenler

5. ConfigurationReader

6. Cache Mekanizması

7. Periyodik Yenileme (Polling)

8. Message Broker (RabbitMQ)

9. Admin Panel

10. Demo API

11. Seed Data

12. Docker Orchestration

13. Test Stratejisi

14. Hata Yönetimi

15. Design Patterns

16. Teknik Kararlar

17. Gereksinim Matrisi

1. Proje Genel Bakışı

Bu proje, uygulama restart veya deployment gerektirmeden konfigürasyon değişikliklerinin runtime'da dinamik olarak yönetilmesini sağlayan production-ready bir .NET 8 sistemidir.

Temel Özellikler

- Dinamik Konfigürasyon:** Deployment veya restart olmadan runtime güncellemeler
- Service Isolation:** Her servis sadece kendi konfigürasyon kayıtlarını görür
- Tip Güvenliği:** `GetValue<T>()` ile generic, strongly-typed erişim
- Thread Güvenliği:** ConcurrentDictionary ile eşzamanlı erişim
- Otomatik Yenileme:** BackgroundService + PeriodicTimer (30sn varsayılan)
- Graceful Degradation:** Storage erişilemezse son başarılı cache ile çalışmaya devam
- Message Broker:** RabbitMQ ile anlık cache yenileme (polling'i tamamlar)
- Otomatik Seed Data:** İlk çalıştırmada demo verileri otomatik eklenir

Teknoloji Yığını

Katman	Teknoloji	Amaç
Runtime	.NET 8	Target framework
Storage	MongoDB 7.0	Konfigürasyon verilerinin saklanması
Cache	ConcurrentDictionary	O(1) hızlı okuma
Message Broker	RabbitMQ 3.x	Anlık bildirim

Web	ASP.NET Core MVC	Admin Panel
API	ASP.NET Core Web API	Swagger'lı Demo API
Logging	Serilog	Yapılandırılmış loglama
Test	xUnit + Moq + FluentAssertions	39 unit test
Container	Docker + Docker Compose	Orchestration

2. Hızlı Başlangıç

Tek Gereklilik: Docker Desktop

Proje, .NET SDK veya herhangi bir ekstra araç gerektirmez. Sadece Docker Desktop yeterlidir.

Kurulum

```
# 1. Repoyu klonla
git clone https://github.com/umuterdal/dynamic-configuration-syst
cd dynamic-configuration-system

# 2. Container'ları başlat
docker-compose up -d

# 3. Servisleri kontrol et
docker ps
```

Servis URL'leri

Servis	URL	Açıklama
Admin Panel	http://localhost:5000	CRUD yönetimi
Demo API	http://localhost:5001/swagger	Swagger UI
RabbitMQ	http://localhost:15672	Management UI (guest/guest)
MongoDB	localhost:27017	Database connection

Durdurma

```
# Container'ları durdur
docker-compose down
```

```
# Volume'larla birlikte sıfırla (veritabanını temizle)
docker-compose down -v
```

Otomatik Seed Data

İlk çalıştırmada Admin Panel otomatik olarak 5 demo konfigürasyon kaydı ekler. Veritabanını sıfırlamak için `docker-compose down -v` komutunu kullanın.

3. Mimari Yapı

Proje, Clean Architecture prensiplerine uygun olarak tasarlanmıştır. Her katman sadece bir altındaki katmana bağımlıdır.

Mimari Diyagram

Configuration.Library (Public API - ConfigurationReader)
Configuration.Application (Business Logic - ConfigurationService, Broker)
Configuration.Infrastructure (MongoDB Repository, Seed Data Service)
Configuration.Domain (Entities, Interfaces, DTOs, Settings)

Proje Yapısı

Proje	Tür	Sorumluluk
Configuration.Domain	Class Library	Entity'ler, interface'ler, DTO'lar, settings. Hiçbir dış bağımlılık yok.
Configuration.Application	Class Library	İş mantığı (ConfigurationService), RabbitMQ publisher
Configuration.Infrastructure	Class Library	MongoDB repository implementasyonu, seed data servisi
Configuration.Library	Class	ConfigurationReader

	Library	(dışarıya açık API), refresh servisi, broker consumer
<code>Configuration.Admin</code>	Web (MVC)	ASP.NET Core MVC Admin Paneli
<code>Configuration.DemoApi</code>	Web (API)	Swagger'lı Demo API
<code>Configuration.UnitTests</code>	Test	39 adet xUnit unit test

Dependency Inversion

Bağımlılıklar Domain katmanına doğru yönelir:

- **Domain:** Hiçbir dış bağımlılık yok (sadece MongoDB.Bson)
- **Application:** Sadece Domain'e bağımlı
- **Infrastructure:** Domain interface'lerini implemente eder
- **Library:** Tüm katmanları orkestra eder

4. Bileşenler

4.1 ConfigurationRecord (Domain Entity)

Her konfigürasyon kaydını temsil eden entity:

Alan	Tip	Açıklama
Id	string (ObjectId)	Unique identifier
Name	string	Konfigürasyon key adı (ör: "SiteName")
Type	string	Veri tipi: "string", "int", "double", "bool"
Value	string	Değer (tip güvenliği için string olarak saklanır)
IsActive	int	1=aktif, 0=pasif. Sadece aktif kayıtlar döner.
ApplicationName	string	Servis izolasyonu için uygulama adı

4.2 ConfigurationRepository (Infrastructure)

MongoDB üzerinde CRUD işlemleri implemente eder. İki constructor'ı vardır:

- `MongoConfigurationRepository(IMongoCollection, ILogger)` — `ConfigurationReader` için
- `MongoConfigurationRepository(IOptions<ConfigurationSettings>, ILogger)` — DI için

Oluşturulduğunda `ApplicationName` üzerinde indeks oluşturarak

performansı artırır.

4.3 ConfigurationService (Application)

İş mantığı katmanı. CRUD işlemlerini doğrulama ve validasyon ile yönetir:

- Geçerli tipler: `string` , `int` , `double` , `bool`
- Paylaşımlı `ValidateRequest()` methodları ile DRY prensibi
- Her CRUD işleminden sonra RabbitMQ'ya change event'i publish eder

5. ConfigurationReader

Kütüphanenin dışarıya açık API'si. 3 parametre ile initialize edilir:

```
var reader = new ConfigurationReader(  
    applicationName: "SERVICE-A",  
    connectionString: "mongodb://localhost:27017",  
    refreshTimerIntervalInMs: 30000);
```

Parametreler

Parametre	Açıklama
<code>applicationName</code>	Uygulama adı. Servis izolasyonu için kullanılır.
<code>connectionString</code>	MongoDB bağlantı adresi.
<code>refreshTimerIntervalInMs</code>	Periyodik yenileme aralığı (milisaniye).

Public Methodlar

`T GetValue<T>(string key)`

Belirtilen key için değeri tip güvenli olarak döner. Supports: `string` , `int` , `double` , `bool` .

```
string siteName = reader.GetValue<string>("SiteName"); // "s  
int maxItems = reader.GetValue<int>("MaxItemCount"); // 50  
double taxRate = reader.GetValue<double>("TaxRate"); // 0.  
bool isEnabled = reader.GetValue<bool>("IsBasketEnabled"); // tru
```

`bool TryGetValue<T>(string key, out T? value)`

Key mevcutsa değeri döner, yoksa false. Exception fırlatmaz.

```
if (reader.TryGetValue<string>("SiteName", out var name))  
    Console.WriteLine(name);
```

IReadOnlyDictionary<string, string> GetAllValues()

Tüm aktif konfigürasyonları dictionary olarak döner.

Task RefreshAsync()

Cache'i manuel olarak yeniler. Eşzamanlı çağrılarda coalesce edilir (interlocked guard).

object GetHealthInfo()

Cache durumu hakkında bilgi döner: son yenileme zamanı, cache yaşı, polling aralığı, cache'lenen key'ler.

Önemli

ConfigurationReader constructor'da MongoDB'ye bağlanıp ilk yükleme yapar. Bu nedenle constructor synchronous olarak `LoadConfigurationsAsync().GetAwaiter().GetResult()` çağırır.

6. Cache Mekanizması

Neden ConcurrentDictionary?

- **O(1)** ortalama okuma süresi
- **Thread-safe** — built-in senkronizasyon
- **Lock-free reads** — çoğu operasyonda kilit yok
- **Sıfır dış bağımlılık** — MemoryCache'e gerek yok

Cache Güncelleme Stratejisi

Yenileme sırasında atomic swap kullanılır:

```
// 1. Yeni cache oluştur
var newCache = new ConcurrentDictionary<string, ConfigurationRecd

// 2. Veritabanından taze verileri yükle
foreach (var record in records)
    newCache[record.Name] = record;

// 3. Atomic swap – referansı değiştir, kilit yok
Interlocked.Exchange(ref _cache, newCache);
```

Eşzamanlılık Güvenliği

- **Single writer:** Yenileme işlemini sadece bir BackgroundService yapar
- **Multiple readers:** Eşzamanlı okuma işlemleri ConcurrentDictionary tarafından yönetilir
- **Overlapping refresh prevention:** `Interlocked.CompareExchange` ile eşzamanlı yenilemeler engellenir

Case-Insensitive Key Erişimi

Cache, `StringComparer.OrdinalIgnoreCase` ile oluşturulur. Bu sayede "sitename" ve "SiteName" aynı key olarak tanınır.

7. Periyodik Yenileme (Polling)

`ConfigurationRefreshService` , `BackgroundService` kullanarak periyodik olarak cache'i yeniler.

Neden BackgroundService + PeriodicTimer?

Yaklaşım	Artıları	Eksileri
BackgroundService + PeriodicTimer Seçilen	Built-in yaşam döngüsü, graceful shutdown, DI entegrasyonu	—
<code>System.Threading.Timer</code>	Daha fazla kontrol	Orphaned callback riski, manuel yaşam döngüsü
Hangfire	Dashboard, persistence	Over-engineered, ek bağımlılık

Implementasyon

```
protected override async Task ExecuteAsync(CancellationToken stop)
{
    using var timer = new PeriodicTimer(_refreshInterval);

    while (await timer.WaitForNextTickAsync(stoppingToken))
    {
        try
        {
            await _configurationReader.RefreshAsync();
        }
        catch (Exception ex)
        {
        }
    }
}
```

```
        _logger.LogError(ex, "Refresh failed");  
    }  
}  
}
```

Polling Nasıl Test Edilir?

1. **Cache durumunu kontrol et:** `curl http://localhost:5001/api/demo/health`
2. **Admin Panel'den değeri değiştir** (ör: SiteName: "soty.io" → "yeni-site.com")
3. **Cache hemen değişmez** (30sn polling süresi var)
4. **Manuel refresh ile anında test et:** `curl -X POST http://localhost:5001/api/demo/refresh`
5. **Veya 30sn bekle** → otomatik yenilenir

8. Message Broker (RabbitMQ)

RabbitMQ, polling'i **tamamlar**, **yerine geçmez**. polling ana mekanizmadır, broker sadece gecikmeyi azaltır.

Neden Her İki Mekanizma?

Mekanizma	Rolü	Gecikme
Polling	Ana mekanizma — her 30sn'de bir kontrol	0-30sn
RabbitMQ	İyileştirme — değişiklik anında bildirim	~100ms

Neden İkisi Birden?

- **Polling tek başına yeterli** — broker çökse bile sistem çalışır
- **Broker tek başına yetersiz** — bağlantı koparsa polling kesintisiz devam eder
- **Production gerçeği:** Hiçbir message broker %100 güvenilir değildir. Polling fallback olarak her zaman çalışır.

Mimari

- **Publisher:** `RabbitMqConfigurationBrokerPublisher` — Admin CRUD sonrası change event'i publish eder
- **Consumer:** `ConfigurationBrokerConsumer` — Event'i alır, anında cache refresh yapar
- **Exchange:** Fanout exchange — tüm consumer'lara bildirim gider
- **Ack:** Manuel acknowledgment — message işlendikten sonra onaylanır

Graceful Degradation

RabbitMQ çökse bile:

- Polling devam eder

- Publisher hatası uygulamayı bozmaz (try-catch ile sarılmış)
- Consumer bağlantı hatası loglanır, polling fallback olarak çalışır

9. Admin Panel

ASP.NET Core MVC ile yazılmış, Bootstrap tabanlı yönetim paneli.

Özellikler

- **Listeleme:** Tüm konfigürasyon kayıtlarını tablo halinde gösterir
- **Oluşturma:** Yeni konfigürasyon kaydı ekler
- **Güncelleme:** Mevcut kayıtları düzenler
- **Silme:** Kayıtları siler (onay ile)
- **Client-side Filtreleme:** Name sütununa göre anlık filtreleme
- **Anti-Forgery:** CSRF koruması

Client-Side Filtreleme

Case gereksinimi: "Kayıtları client side olarak ismi ile filtrelenebilmelidir."

```
// JavaScript – keyup event ile Name sütununda filtreleme
document.getElementById('searchInput').addEventListener('keyup',
    var filter = this.value.toLowerCase();
    var rows = document.querySelectorAll('#configurationTable tr')
    rows.forEach(function (row) {
        if (row.cells.length < 6) return;
        var name = row.cells[1]?.textContent.toLowerCase() || '';
        row.style.display = name.includes(filter) ? '' : 'none';
    });
});
```

Change Event Publishing

Her CRUD işleminden sonra RabbitMQ'ya change event'i publish edilir:

- Create → `PublishAsync(applicationName, "Created")`
- Update → `PublishAsync(applicationName, "Updated")`
- Delete → `PublishAsync(applicationName, "Deleted")`

10. Demo API

Swagger UI ile donatılmış, ConfigurationReader kullanımını gösteren REST API.

Endpoint'ler

Method	Endpoint	Açıklama
GET	/api/demo/value/{key}	String değer getir
GET	/api/demo/typed/{key}? type=int	Tip belirterek değer getir
GET	/api/demo/all	Tüm konfigürasyonları getir
GET	/api/demo/health	Cache durumu ve polling bilgisi
POST	/api/demo/refresh	Cache'i yenile

Health Endpoint Örneği

```
{
  "applicationName": "SERVICE-A",
  "lastCacheUpdate": "2026-07-09T16:30:21Z",
  "cacheAgeSeconds": 25,
  "refreshIntervalMs": 30000,
  "cachedKeyCount": 4,
  "cachedKeys": ["TaxRate", "MaxItemCount", "IsBasketEnabled",
  "mongoDB": "healthy"
}
```

Admin Panel Health Endpoint

Admin Panel footer'ında MongoDB durumu gösterilir:

```
# Admin Panel health endpoint
curl http://localhost:5000/Configuration/Health

# Response:
{
  "applicationName": "Configuration.Admin",
  "lastCacheUpdate": "2026-07-09T18:03:38Z",
  "cacheAgeSeconds": 22,
  "refreshIntervalMs": 30000,
  "cachedKeyCount": 0,
  "mongoDB": "healthy"
}
```

Footer'da **MongoDB** yazısı yeşil (sağlıklı) veya kırmızı (sağlıksız) olarak gösterilir, yanında tik/x ikonu bulunur.

Örnek Kullanım

```
# Tüm konfigürasyonları getir
curl http://localhost:5001/api/demo/all

# Tek değer getir
curl http://localhost:5001/api/demo/value/SiteName

# Tip belirterek getir
curl "http://localhost:5001/api/demo/typed/TaxRate?type=double"

# Cache yenile
curl -X POST http://localhost:5001/api/demo/refresh

# Sağlık durumu
curl http://localhost:5001/api/demo/health
```

11. Seed Data

`SeedDataService` , uygulama ilk başladığında veritabanı boşsa demo verilerini otomatik olarak ekler.

Neden Seed Data?

- İndiren kişi hemen sistemi test edebilmeli
- Manuel veri ekleme zahmetinden kurtulmalı
- Admin Panel ve Demo API hemen kullanılabilir olmalı

Seed Data İçeriği

Name	Type	Value	IsActive	ApplicationName
SiteName	string	soty.io	1 (aktif)	SERVICE-A
MaxItemCount	int	500	1 (aktif)	SERVICE-A
TaxRate	double	0.18	1 (aktif)	SERVICE-A
IsBasketEnabled	bool	true	1 (aktif)	SERVICE-A
MaintenanceMode	bool	false	0 (pasif)	SERVICE-A

Idempotent Tasarım

`SeedDataService` **idempotent**tir — veritabanında veri varsa tekrar eklemes:

```
var existing = await _repository.GetAllAsync(stoppingToken);
if (existing.Count > 0)
{
    _logger.LogInformation("Database already contains {Count} rec
    return;
}
```

Neden Sadece Admin Panel'de?

- Admin CRUD yönetim paneli — veri oradan eklenir/yönetilir
- DemoApi sadece okuyucu — seed yapmasına gerek yok
- İki taraf da seed yaparsa race condition riski olur

Log Çıktısı

İlk çalıştırmada:

```
[INF] Seeding initial configuration data...
```

```
[INF] Successfully seeded 5 configuration records
```

Sonraki çalıştırmalarda:

```
[INF] Database already contains 5 records, skipping seed
```

12. Docker Orchestration

`docker-compose.yml` 4 container ile tam bir ortam sağlar.

Container'lar

Servis	Image	Port	Sağlık Kontrolü
configuration-mongodb	mongo:7.0	27017	<code>mongosh --eval db.adminCommand('ping')</code>
configuration-rabbitmq	rabbitmq:3-management	5672, 15672	<code>rabbitmq-diagnostics check_port_connectivity</code>
configuration-admin	Özel build	5000	<code>curl -f http://localhost:8080/</code>
configuration-demoapi	Özel build	5001	<code>curl -f http://localhost:8080/swagger/index.html</code>

Dependency Zinciri

- Admin ve DemoApi, **MongoDB healthy** olduktan sonra başlar
- Admin ve DemoApi, **RabbitMQ healthy** olduktan sonra başlar
- `depends_on` ile `condition: service_healthy` kullanılır

Environment Variables

Configuration bilgileri environment variable ile verilir (12-factor app prensibi):

```
environment:  
  - ConfigurationReader__ConnectionString=mongodb://admin:password  
  - ConfigurationReader__ApplicationName=SERVICE-A  
  - ConfigurationReader__DatabaseName=ConfigurationDb
```

- ConfigurationReader__CollectionName=Configurations
- ConfigurationReader__RefreshTimeIntervalInMs=30000
- ConfigurationBroker__HostName=rabbitmq
- ConfigurationBroker__Port=5672
- ConfigurationBroker__ExchangeName=config-changes

Multi-Stage Build

Dockerfile'lar multi-stage build kullanır:

1. **Restore:** NuGet paketlerini indir
2. **Build:** Proje derleme
3. **Publish:** Publish işlemi
4. **Final:** Sadece gerekli dosyalar ile küçük image

13. Test Stratejisi

39 adet xUnit unit test ile kapsamlı test coverage.

Test Araçları

Araç	Amaç
xUnit	Test framework
Moq	Mock nesneler
FluentAssertions	Okunabilir assertions

Test Kategorileri

ConfigurationReader Tests (16 test)

- GetValue — string, int, double, bool tip dönüşümleri
- GetValue — eksik key hatası (KeyNotFoundException)
- GetValue — pasif key hatası
- GetValue — geçersiz tip dönüşümü hatası
- GetValue — case-insensitive erişim
- TryGetValue — mevcut/olmayan key
- GetAllValues — sadece aktif kayıtlar
- RefreshAsync — cache güncelleme
- Constructor — null parametreler
- Constructor — geçersiz refresh aralığı
- Dispose sonrası erişim

ConfigurationService Tests (10 test)

- GetAllConfigurationsAsync — tüm kayıtlar
- GetConfigurationsByApplicationAsync — filtreleme

- GetByIdAsync — mevcut/olmayan ID
- CreateAsync — geçerli/geçersiz istek
- UpdateAsync — güncelleme
- DeleteAsync — silme

Broker Tests (13 test)

- RabbitMQ Publisher — 6 test (publish, null parametreler, hata senaryoları)
- Broker Consumer — 7 test (mesaj işleme, constructor validasyonu, graceful degradation)

Test Çalıştırma

```
# Tüm testleri çalıştır  
dotnet test
```

```
# Coverage ile çalıştır  
dotnet test --collect:"XPlat Code Coverage"
```

14. Hata Yönetimi

Graceful Degradation

Storage erişilemezse sistem son başarılı cache ile çalışmaya devam eder:

1. **İlk yükleme başarısız:** Boş cache — uygulama başlar ama veri yok
2. **Yenileme başarısız:** Önceki cache korunur
3. **Uygulama çalışmaya devam eder** — eski verilerle
4. **Warning logları** — monitoring için

Implementasyon

```
private async Task RefreshConfigurationsAsync()
{
    try
    {
        var records = await _repository.GetByApplicationNameAsync();
        var newCache = new ConcurrentDictionary<string, Configuration>();
        foreach (var record in records)
        {
            newCache[record.Name] = record;
        }
        Interlocked.Exchange(ref _cache, newCache);
    }
    catch (Exception ex)
    {
        // HATA: Cache'deki önceki değerleri KORU
        _logger.LogWarning(ex,
            "Failed to refresh. Cache will retain previous values");
    }
}
```

RabbitMQ Hata Yönetimi

- Publisher: Hata durumunda exception fırlatmaz, log yazar
- Consumer: `BasicNack` ile mesajı tekrar kuyruğa atar
- Bağlantı kopması: `AutomaticRecoveryEnabled = true` ile otomatik

yeniden bağlanma

Exponential Backoff

Yenileme denemelerinde üstel gecikme stratejisi kullanılır:

- İlk deneme: anında
- 2. deneme: 500ms bekleme
- 3. deneme: 1000ms bekleme
- Maksimum 3 deneme — başarısız olursa cache korunur

15. Design Patterns

Kullanılan Patterns

Pattern	Nerede Kullanıldı	Neden?
Clean Architecture	Tüm proje yapısı	Sorumlulukların ayrılması, test edilebilirlik
Repository Pattern	IConfigurationRepository	Data access soyutlaması
Dependency Injection	Tüm servisler	Loosely coupled, test edilebilir
Options Pattern	ConfigurationSettings, BrokerSettings	Tipli konfigürasyon yönetimi
Background Service	ConfigurationRefreshService, SeedDataService, BrokerConsumer	Periyodik işlemler
Service Layer	ConfigurationService	İş mantığı izolasyonu

SOLID Prensipleri

- **S (Single Responsibility):** Her sınıfın tek bir sorumluluğu var
- **O (Open/Closed):** Yeni storage eklemek için repository interface'ini implemente etmek yeterli
- **L (Liskov Substitution):** Repository mock ile test edilebilir
- **I (Interface Segregation):** Küçük, odaklı interface'ler
- **D (Dependency Inversion):** Bağımlılıklar abstract interface'lere

16. Teknik Kararlar

MongoDB vs SQL Server vs Redis

Kriter	MongoDB	SQL Server	Redis
Schema esnekliđi	✓ Yüksek	✗ Düşük	✓ Yüksek
Sorgulama gücü	✓ İyi	✓ En iyi	✗ Sınırlı
Deployment kolaylıđı	✓ Docker image	✗ Lisans	✓ Docker image
Persistence	✓ Kalıcı	✓ Kalıcı	⚠ Volatile
Scalability	✓ Sharding	⚠ Zor	✓ Cluster

Karar: MongoDB — esnek schema, kolay deployment, iyi .NET desteđi.

ConcurrentDictionary vs MemoryCache

Kriter	ConcurrentDictionary	MemoryCache
Dış bağımlılık	✓ Yok	✗ Microsoft.Extensions.Caching
Control	✓ Tam kontrol	⚠ Framework'e bağımlı
Otomatik temizleme	✗ Manuel	✓ Built-in
Performans	✓ O(1)	✓ O(1)

Karar: ConcurrentDictionary — sıfır bağımlılık, tam kontrol.

BackgroundService vs Timer

Karar: BackgroundService + PeriodicTimer — graceful shutdown, DI entegrasyonu.

Polling + Broker (Neden İki?)

- **Polling tek başına yeterli** — broker çökse bile çalışır
- **Broker tek başına yetersiz** — bağlantı koparsa polling devam eder
- **Production gerçeği:** Hiçbir broker %100 güvenilir değildir

17. Gereksinim Matrisi

Case dokümanındaki tüm gereksinimler ve durumları:

Temel Gereksinimler

#	Gereksinim	Durum	Implementasyon
1	Konfigürasyon kaydı yapısı (Id, Name, Type, Value, IsActive, ApplicationName)	✓	ConfigurationRecord.cs
2	.NET 8 kütüphanesi (DLL)	✓	Configuration.Library — net8.0
3	3 parametre ile initialize	✓	ConfigurationReader(appName, connStr, interval)
4	T GetValue<T>(string key) generic method	✓	ConfigurationReader.cs
5	string, int, double, boolean tip desteği	✓	ConvertValue<T>()
6	Service isolation (her servis sadece kendi kayıtlarını görür)	✓	GetByApplicationNameAsync()
7	Yalnızca IsActive=1 kayıtları döner	✓	Repository + Reader filtresi
8	Periyodik yenileme (parametrik süre)	✓	ConfigurationRefreshService + PeriodicTimer

9	Storage erişilemezse son başarılı cache ile devam	✓	Graceful degradation
---	---	---	----------------------

Web Arayüzü

#	Gereksinim	Durum	Implementasyon
10	Kayıtları listeleme	✓	Admin Panel Index
11	Kayıt güncelleme	✓	Admin Panel Edit
12	Yeni kayıt ekleme	✓	Admin Panel Create
13	Client-side Name filtreleme	✓	JavaScript keyup event

Ekstra Puan

#	Kriter	Durum	Implementasyon
14	Message Broker (RabbitMQ)	✓	Publisher + Consumer
15	TPL, async/await	✓	Tüm asenkron işlemler
16	Concurrency handling	✓	ConcurrentDictionary + Interlocked
17	Design & Architectural Patterns	✓	Clean Architecture, SOLID, Repository, DI
18	TDD	✓	Test-driven development
19	Unit Tests	✓	39 test — xUnit + Moq
20	MongoDB	✓	MongoDB 7.0

21	Çalışır Proje	✓	docker-compose up -d
22	Dokümantasyon	✓	README.md + Documentation.html
23	Source Control	✓	GitHub
24	Docker Compose	✓	4 container

Sonuç

Tüm gereksinimler (24/24) karşılanmıştır. Ekstra puan kriterlerinin tamamı uygulanmıştır.

Dynamic Configuration System — Teknik Dokümantasyon

.NET 8 | MongoDB | RabbitMQ | Clean Architecture | Docker